

CASE STUDY

Sat-Raj

A fuel distribution platform that replaced 30 years of spreadsheets for a NJ wholesaler.

ROLE

Solo full-stack engineer (contract)

LIVE AT

satraj.inc

STATUS

Live · Jan – Apr 2026 · Client: Sat-Raj Inc.

STACK

Next.js 16 · React 19 · TypeScript · Prisma 6 · PostgreSQL · NextAuth v5 · AWS (Amplify, RDS, SES, S3) · Samsara API · DTN · QuickBooks

Uzair Saleem · uzairsaleem.dev

The brief

TL;DR

Sat-Raj had been running a 30-year-old fuel distribution business out of **39 hand-edited Google Sheets tabs**, a master tax/margin template, and a freight matrix — updated by hand every afternoon, then emailed to 24 customers one at a time. I replaced it with a multi-tenant platform that pulls supplier costs from the DTN feed, calculates daily prices, distributes them in one click, ingests bills of lading from Samsara, and generates customer invoices.

The daily pricing run that took **45–60 minutes** of spreadsheet work and email copy/paste now takes **under 90 seconds** — with full audit history and zero per-customer formula drift.

The problem

Sat-Raj buys fuel from Phillips 66 and the Linden terminal, marks it up for tax, freight, and margin, and resells to ~24 gas stations across NJ and PA. Every afternoon at ~6:00 PM, prices need to go out so customers can plan the next day's orders.

The inherited workflow:

- Suppliers email terminal prices.
- Operations types new costs into a "Price Template" sheet.
- Formulas across 39 customer tabs recalculate sell prices ($\text{cost} + \text{tax} + \text{freight} + \text{margin}$).
- Each tab gets manually copied into an email and sent.
- BOLs from Samsara screenshots get re-typed into Sheets, then again into QuickBooks.

Pain points exactly what you'd expect from a spreadsheet stack at this scale:

- **Bus-factor of 1.** Only one person fully understood the formula chain.
- **Per-customer drift.** Tabs hand-edited over years; tax rules and margins quietly diverged.
- **No audit trail.** Disputed prices were resolved by digging through Gmail's Sent folder.
- **Multi-state complexity.** NJ and PA have different tax structures across fuel types.
- **Multi-location customers.** Per-location freight rates modeled inconsistently across tabs.
- **Triple data entry downstream.** Samsara → Sheets → QuickBooks.

What I built

Pricing engine

The core unlock. I modeled the spreadsheet's implicit data structure as first-class entities: **Suppliers** with daily terminal prices (auto-ingested from the DTN feed), **tax rules** scoped by state + fuel family, **margins** per fuel type, **clients** with multi-location support and per-location freight rates, and **price runs** as immutable snapshots — every email sent is reproducible from stored inputs.

The UI is a 3-step stepper: enter today's costs → review the calculated grid for all 24 customers → send. Per-customer email templates ship via AWS SES (production access + DKIM verified for satrajinc.com). Pricing math lives in one file (`lib/pricing.ts`) with tests pinned to the legacy sheet's rates to prove parity.

DTN supplier ingestion

The DTN parser pulls supplier price files daily, normalizes them against the supplier/terminal mapping, and writes terminal prices straight into the database. After the first month in production we caught a column-mapping bug where gross and net gallons were swapped at one terminal; I shipped the fix and a one-shot migration to repair the affected historical rows in the same commit window.

BOL pipeline (Samsara → invoices)

When a driver completes a delivery, the Samsara mobile app captures a signed BOL. The platform pulls documents two ways: DTN-sourced BOLs for terminal pickups (gross, net, trailer, account number) and Samsara-sourced BOLs for customer deliveries (signed BOL number, gallons, document URL). Both feed a unified bills-of-lading view, joined to the originating dispatch and the destination client. From there, deliveries flow into a **review queue** where staff confirm site mapping (Samsara geofence → client location) before invoice generation.

Site mapping & review queue

Samsara geofences don't always line up with the client locations in our database — especially for multi-site customers. I built a site-mapping UI and a review queue so anything ambiguous gets flagged for human approval rather than silently misattributed. Once a Samsara address is mapped, the system remembers it via a `location_aliases` table.

Auth, RBAC, settings

NextAuth v5 with bcrypt-hashed credentials and middleware-protected routes. Two roles (`ADMIN` / `TEAM_MEMBER`), admin-only team management, self-service profile changes. Tax rules, margins, and supplier configs are all DB-backed — no hardcoded constants, no deploy required for a tax change.

Technical decisions worth calling out

- **Next.js App Router over a separate API + SPA.** Single deployable surface (Amplify + CloudFront), RSC pushes DB queries straight into pages for read-only views; API routes reserved for mutations and webhooks.
- **Prisma + PostgreSQL on RDS.** Schema-first made the migration from Sheets clean — the schema doc was the alignment artifact with the client. RDS over Aurora because the load profile (24 customers, ~1k deliveries/day at peak) doesn't justify Aurora's cost.
- **Configuration in DB, not env constants.** Tax rates change. Operations needed to update them without a deploy.
- **Replicate the legacy formula exactly, then improve.** First launch had to produce *byte-identical* prices to the spreadsheet on a test day or the client wouldn't trust it. Parity first, validation later.
- **SES over SendGrid.** Already in the AWS account, no extra vendor, DKIM was a one-time setup.
- **Human-in-the-loop on Samsara mapping.** Deliberately did not auto-match geofences on first sight — the review queue catches edge cases before they corrupt invoice data.

Impact

40× Faster daily price run	90 sec From 45–60 min	0 Per-customer drift	24 Customers automated
--------------------------------------	---------------------------------	--------------------------------	----------------------------------

Metric	Before	After
Daily price distribution	45–60 min, manual	Under 90 sec, one-click
Per-customer formula drift	Frequent, undetected	Eliminated (single source of truth)
Price audit history	Search Gmail's Sent folder	Queryable in-app, indefinite
Bus-factor on pricing	1 person	Any team member with login
BOL data entry	Re-typed twice	Pulled once, mapped, invoiced
Supplier price entry	Manual every morning	Automated via DTN parser

Beyond the numbers: the platform unblocked things the business couldn't previously consider — onboarding new staff without a 2-week spreadsheet apprenticeship, real per-delivery profit visibility, and confidence that customer invoices match what was actually delivered.

Stack

Next.js 16	React 19	TypeScript	Tailwind 4	Prisma 6	PostgreSQL
NextAuth v5	AWS Amplify	AWS RDS	AWS SES	AWS S3	Samsara API
DTN feed	QuickBooks				